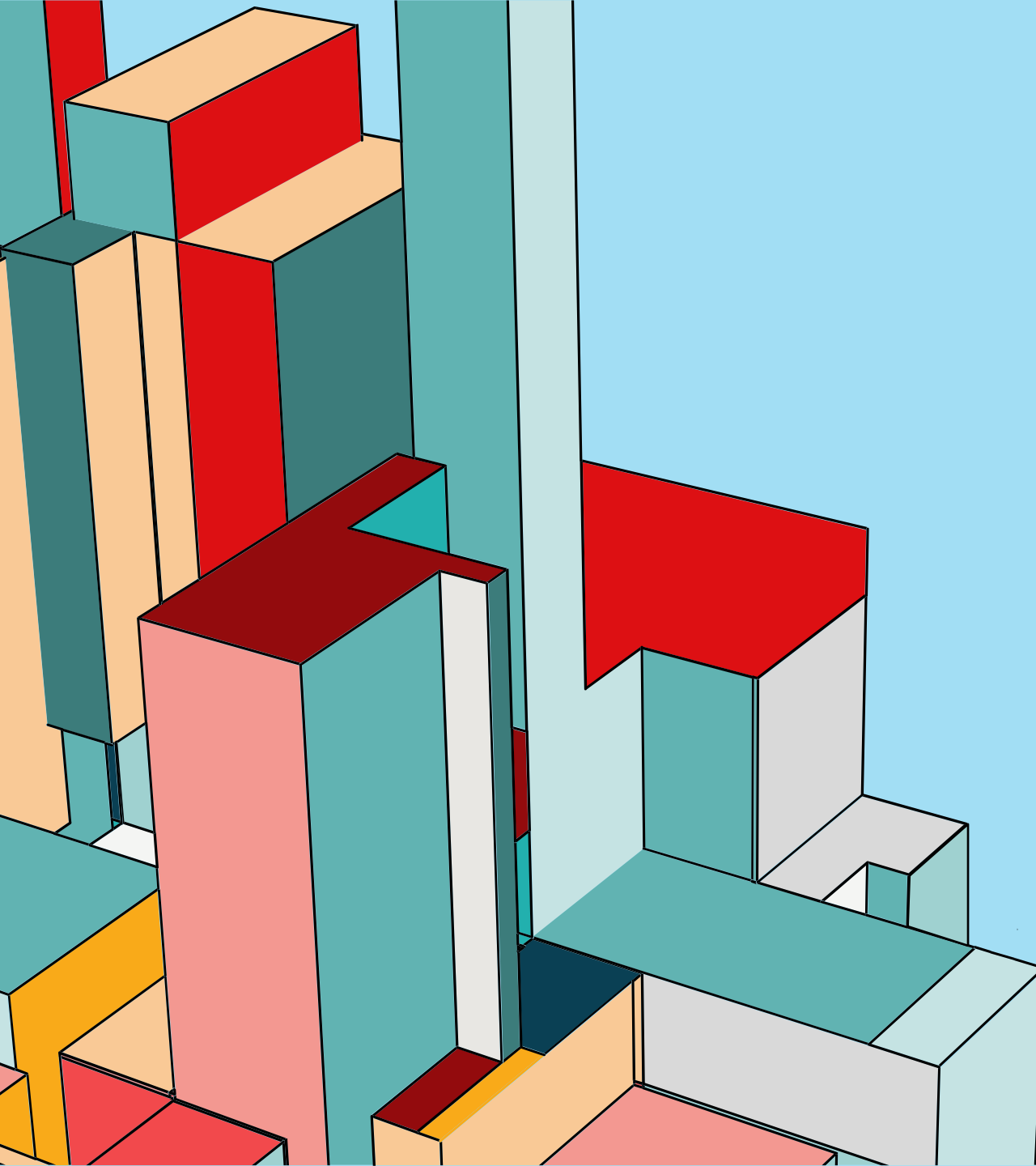
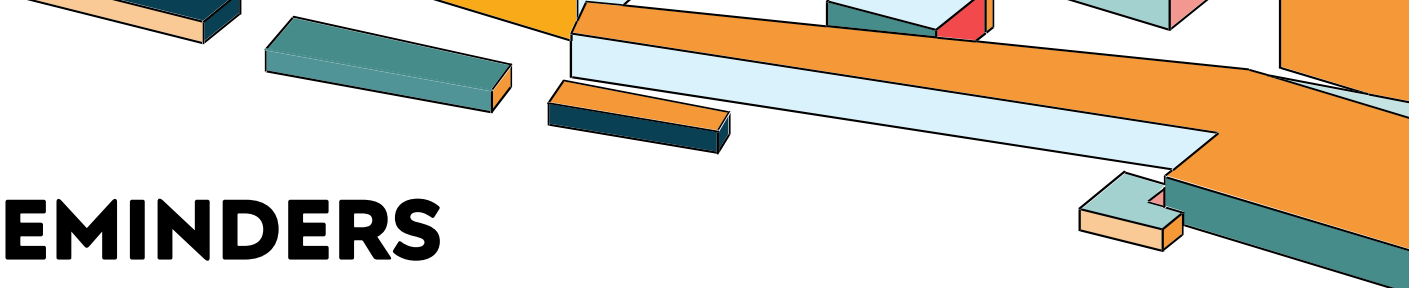




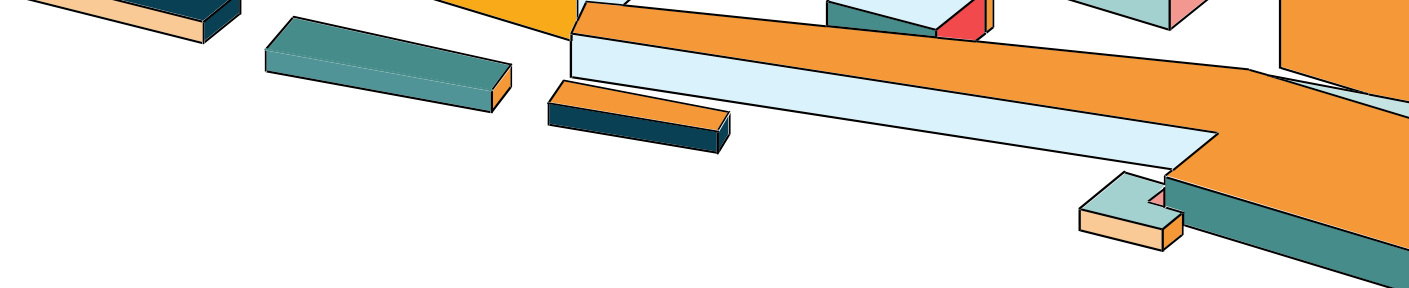
LECTURE 21 **FLOW NETWORKS II**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

A collection of 3D rectangular blocks in various colors (orange, teal, light blue, dark blue, red) arranged in a scattered, overlapping pattern in the top right corner of the slide.

ANNOUNCEMENTS / REMINDERS

- Quantum Hackathon this weekend
- HW 9 due tomorrow; normal lab this week
- Exam 3 next Wednesday 4/15.

A collection of 3D rectangular blocks in various colors (orange, teal, light blue, dark blue) arranged in a scattered pattern in the top right corner of the slide.

AGENDA

- Quick recap of last class
- Applications of flow networks
- Extensions of the basic problem

THE MAXFLOW PROBLEM

A *flow network* is a connected, directed graph with strictly positive edge weights that has a single source and a single sink.

A *flow* is an assignment of a non-negative numeric value to each edge such that (a) the flow on each edge does not exceed the edge weight and (b) at every interior node, flow in is equal to flow out. The *value* of the flow is the total flow out of the source (equivalently, into the sink).

The *Maximum Network Flow* problem (*MaxFlow* for short) asks us to find a flow with maximum value.

THE MINCUT PROBLEM

A *cut* in a graph is a partition of the vertices into two nonempty groups V_1 and V_2 such that $V_1 \cup V_2 = V$ and $V_1 \cap V_2 = \emptyset$. The term *cut* is also used to refer to the edges that have one endpoint in V_1 and one in V_2 (the edges that cross the cut).

A *source-sink cut* (often *s-t cut*) is a cut of a flow network that places the source and sink into different groups. The *capacity* of a cut is the sum of the weights of the edges that cross the cut.

The *Minimum Capacity Cut* problem (*MinCut* for short) asks us to find a source-sink cut with the minimum possible capacity.

DUALITY AND THE FORD-FULKERSON PROCEDURE

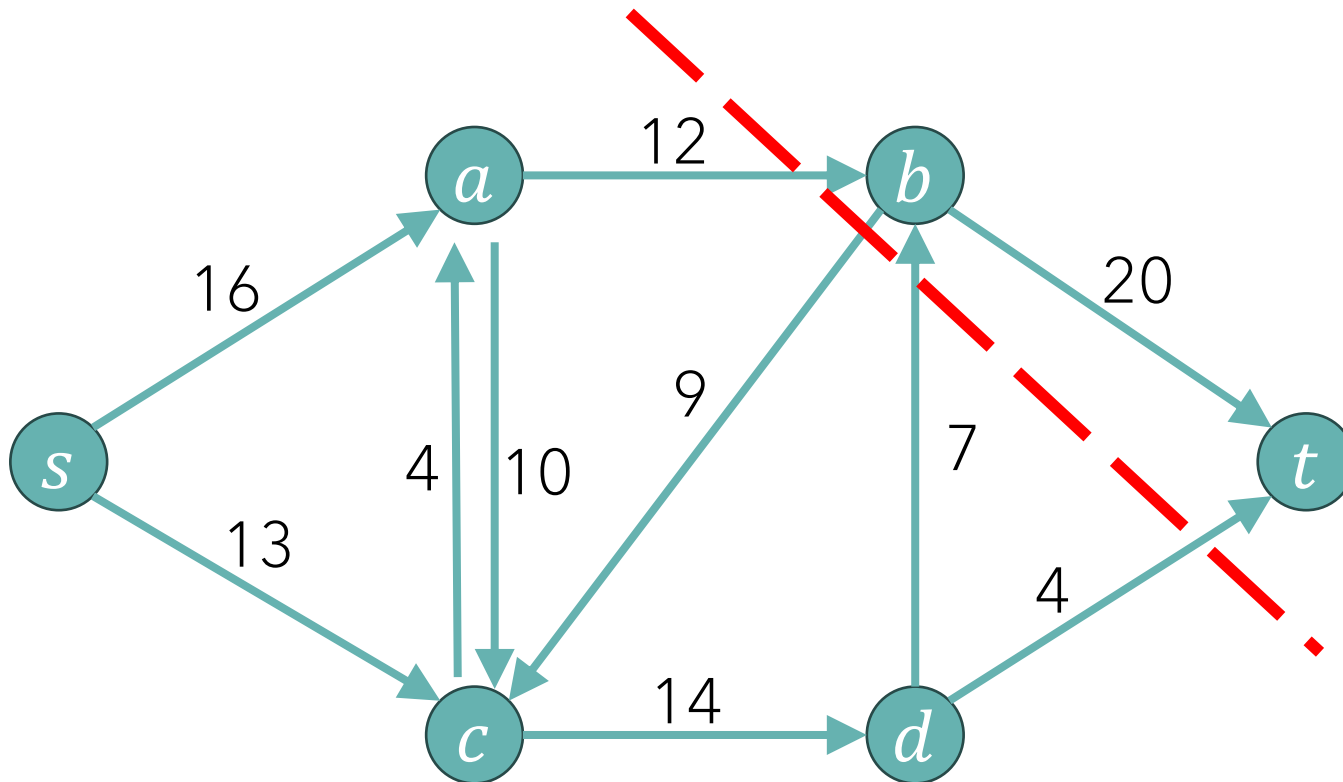
MaxFlow and MinCut are *dual* problems – solutions to each problem place bounds on the best possible result in the other.

In this case, the bounds are tight: Ford & Fulkerson proved that the max flow equals the min cut for every flow network. Their proof is *constructive* – the Ford-Fulkerson Procedure produces a flow with maximum value and a cut with (equal) minimum value.

FFP operates by setting up a *residual graph* that reverse the edges when flow is assigned to them. Paths from s to t in the residual graph represent *augmenting paths* in the original flow network – paths that will increase the flow value. From multiple such paths, we select the one with fewest links (Edmonds-Karp).

PRACTICE: SUBMITTY POLL #1

Find the minimum cut in this graph and provide your answer in the Submitty poll.



TIMELINE OF KEY IMPROVEMENTS ON THE BASIC NETWORK FLOW PROBLEM

- 1956: Ford & Fulkerson publish their procedure. $O(CE)$
- 1970: Dinitz proves that always selecting the shortest augmenting path produces an $O(VE^2)$ algorithm.
- 1986: Goldberg and Tarjan publish the push-relabel algorithm, which uses *preflows* (flows that can temporarily violate conservation by “storing” flow at an internal node) to produce an $O(V^2E)$ algorithm.
- 1994: King, Rao, and Tarjan publish the KRT algorithm, which runs in $O(VE \log V)$ time.
- 2013: Orlin publishes an $O(VE)$ algorithm.

A QUESTION

Q: Why have researchers spent so much effort on finding faster solutions to this one problem?

A: REDUCTION

We can model a LOT of other problems using network flows. A fast solution to this one problem then solves these problems: bipartite matching, disjoint paths, survey design, airline scheduling, image segmentation, tournament elimination, etc...

MATCHING CANDIDATES TO JOBS...

Medical residency programs are essentially apprenticeships for new doctors. Every year, hospitals look for new residents and new medical school graduates look for residency programs.

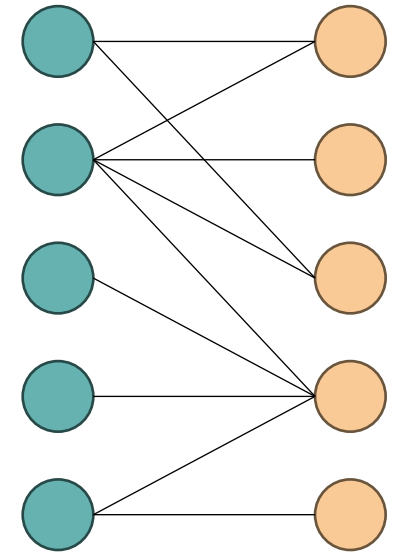
There are, of course, restrictions. Each doctor is only looking for certain specialities. Location of the hospital is a huge factor. Some programs are highly selective and only want top graduates.

How might we model this situation?

BIPARTITE GRAPHS & MATCHING

This is a situation where a *bipartite graph* is the obvious choice. As a reminder, a bipartite graph is one in which we can divide the vertices up into two groups L and R such that every edge connects a vertex in L to a vertex in R . (Put another way, neither L nor R have any internal edges.)

We can make an L vertex for each graduating student and an R vertex for every residency position, with edges connecting the compatible ones.



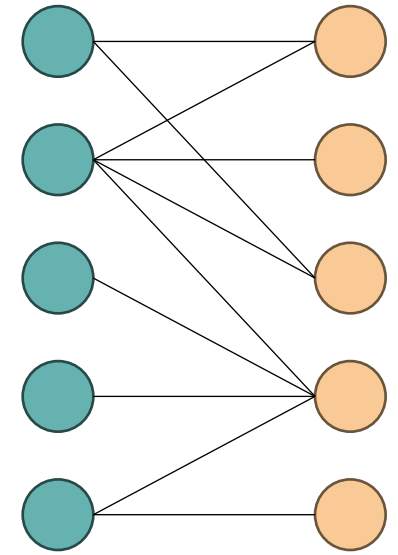
BIPARTITE GRAPHS & MATCHING

Once the graph is constructed, the obvious question becomes: Can we match up every doctor with a position and vice versa?

If not, what is the largest number of matches that we can make?

In any graph, a *matching* is a subset of edges in which each vertex appears as an endpoint at most once. (i.e. matched with one other node, or not).

The *Maximum Bipartite Matching* problem asks us to find the largest possible matching in a given bipartite graph. What's your algorithm?

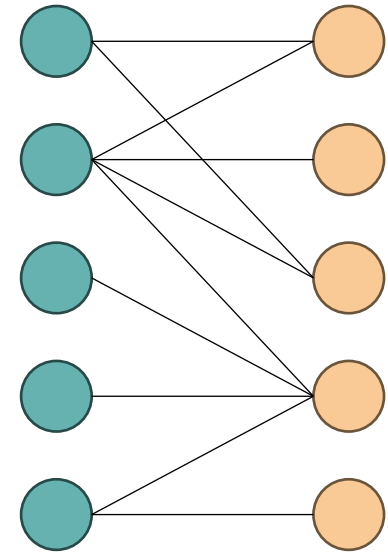


BIPARTITE MATCHING VIA MAXFLOW

So here's a neat idea: Let's turn the bipartite graph into a flow network.

What do we need to do?

- Add directions to the existing edges.
- Add a source, a sink, and edges connecting them to some of the existing vertices.
- Add capacities to all edges.



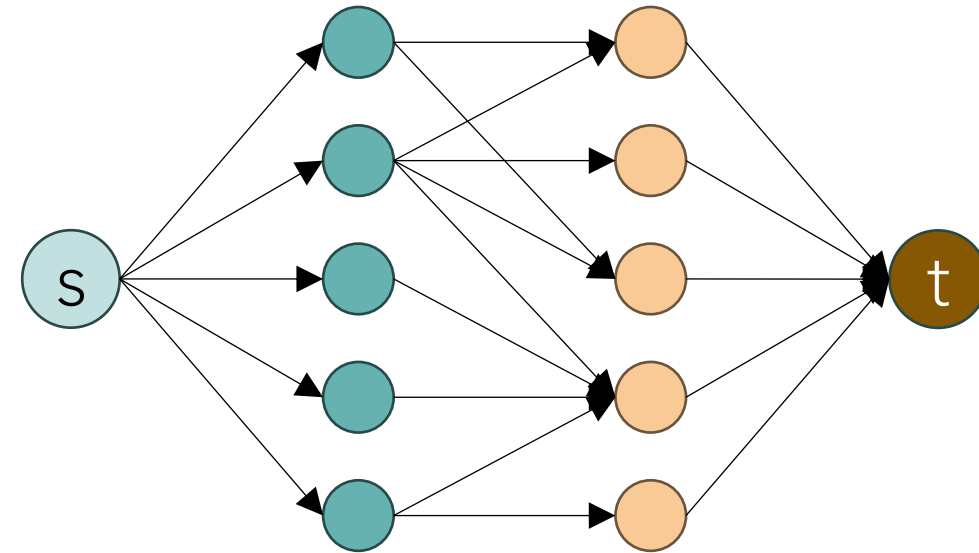
BIPARTITE MATCHING VIA MAXFLOW

- Add directions to the existing edges.

The simplest approach is simply to orient all of the edges from L to R .

- Add a source, a sink, and edges connecting them to some of the existing vertices.

Similarly, we can connect the source to every node in L and every node in R to the sink.

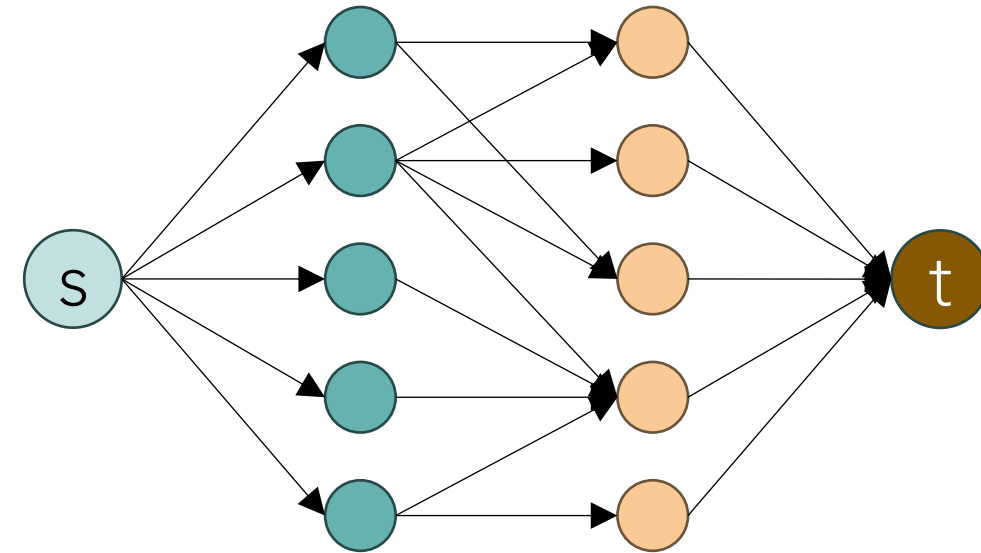


BIPARTITE MATCHING VIA MAXFLOW

- Add capacities to all edges.

The simplest possible approach is just to give every edge a capacity of 1.

It turns out that this is enough! A solution to MaxFlow on this network solves the Maximum Bipartite Matching problem – do you see why?

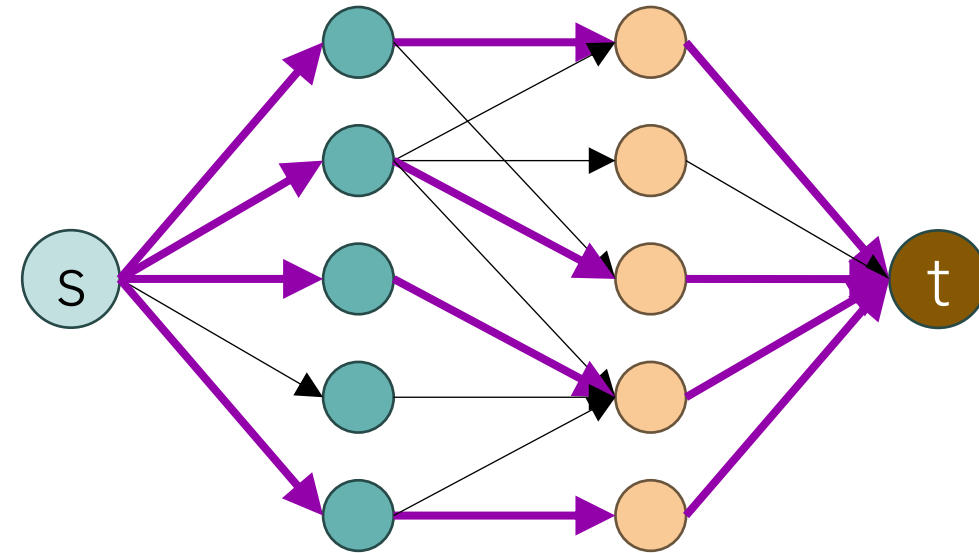


BIPARTITE MATCHING VIA MAXFLOW

Consider any flow in this network. Every unit of flow moves from s , to a vertex of L , to a vertex of R , and finally to t .

Thus, every unit of flow pairs an L node with an R node. Furthermore, those nodes cannot be re-used because the edges from s and to t are at capacity.

The value of the flow is thus the size of the match found. But this does not, by itself, guarantee that the optimal flow is also the optimal match.



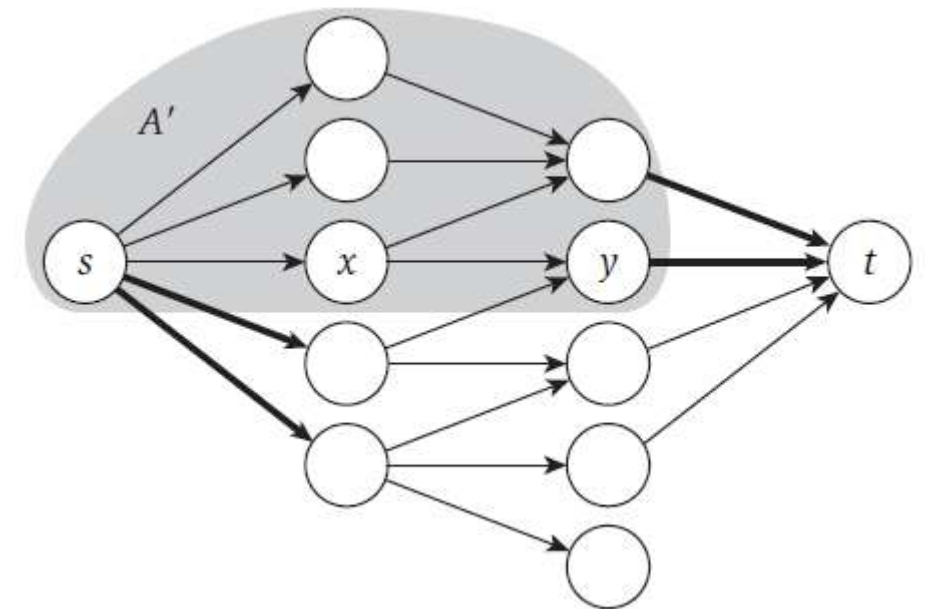
BIPARTITE MATCHING VIA MAXFLOW

The proof of optimality rests upon Hall's Theorem, which is an important result in graph theory. It says:

In a finite bipartite graph, a *perfect matching* exists if and only if, $\forall k$, every set of k vertices from one side has at least k neighbors in the other group.

For example, in the diagram at right, there is a set of three L nodes that is connected to a set of only two R nodes.

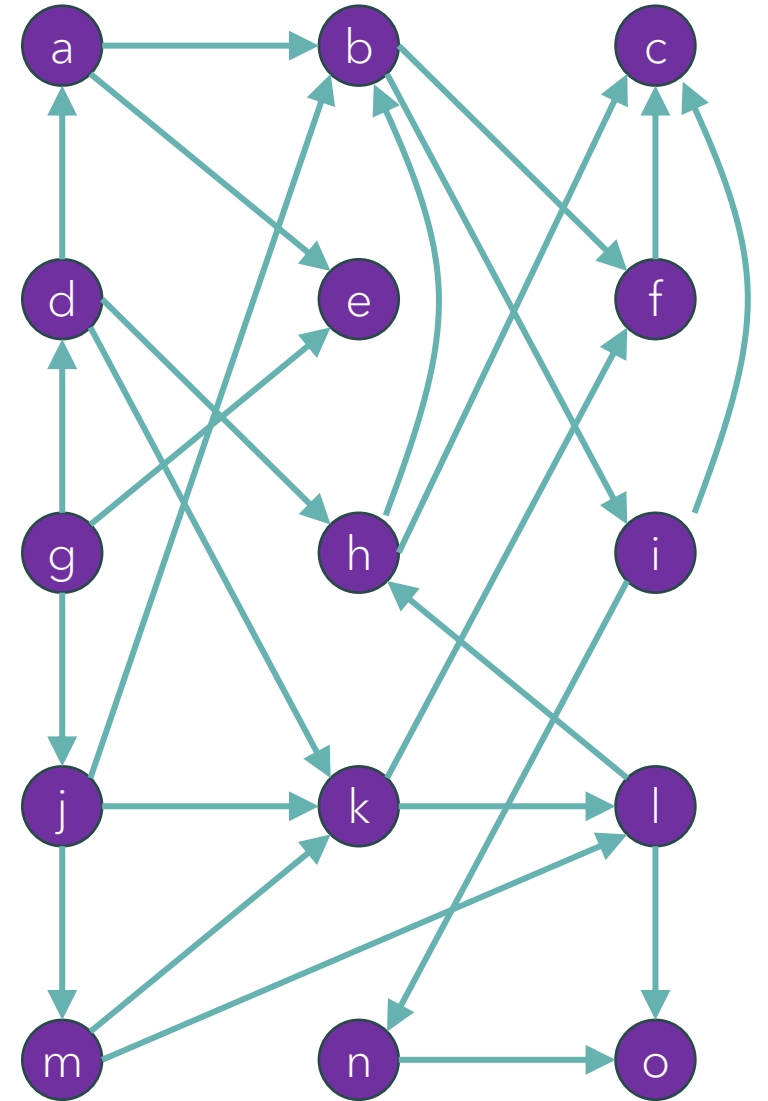
The MinCut of the network will always find such a set if it exists!



THE DISJOINT PATHS PROBLEM

Given a graph G and any two vertices u and v , how many paths from u to v are there such that no edge is shared by two of the paths? (i.e. can't reuse edges)

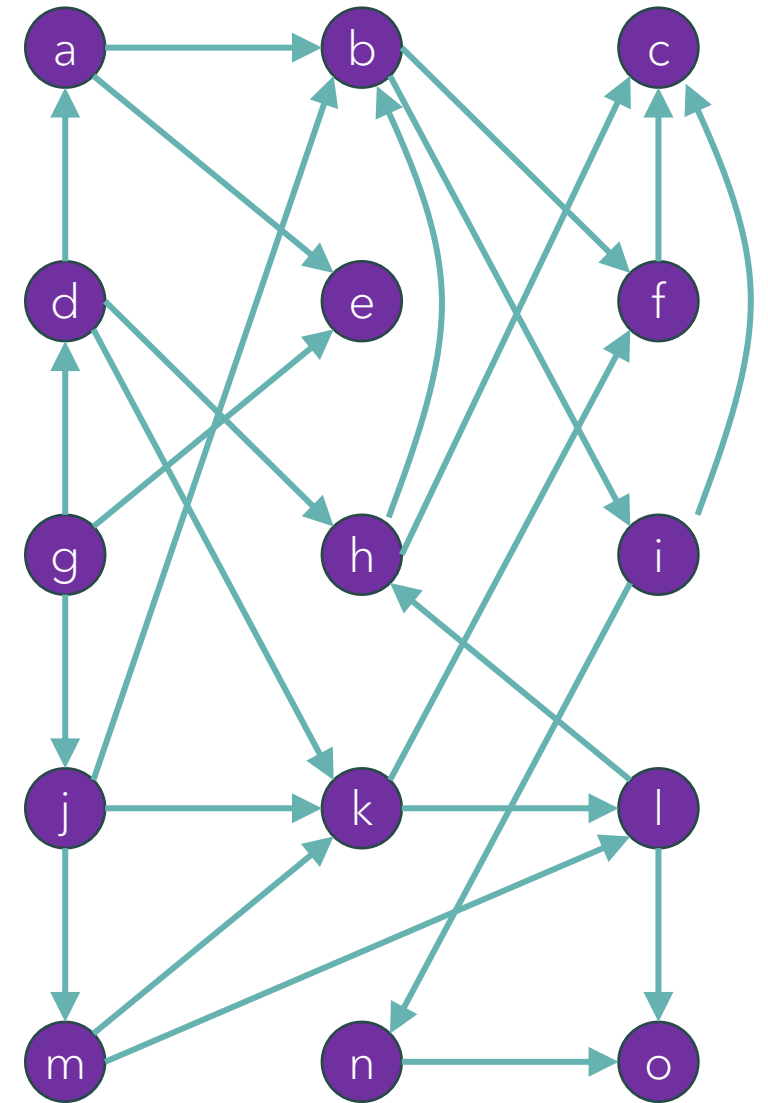
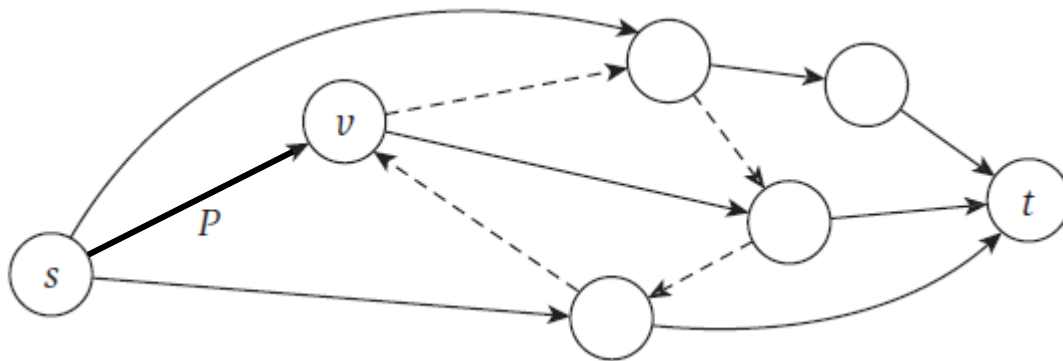
This problem can be solved in basically the same way as the last one: set all capacities to 1, let u be the source and v be the sink. There are a few wrinkles, though...



THE DISJOINT PATHS PROBLEM

If u has incoming edges or v has outgoing edges, we can just add new “dummy” source/sink nodes that only link to u/v .

The larger issue is that a valid flow can have loops. Removing these makes the answer unambiguous.

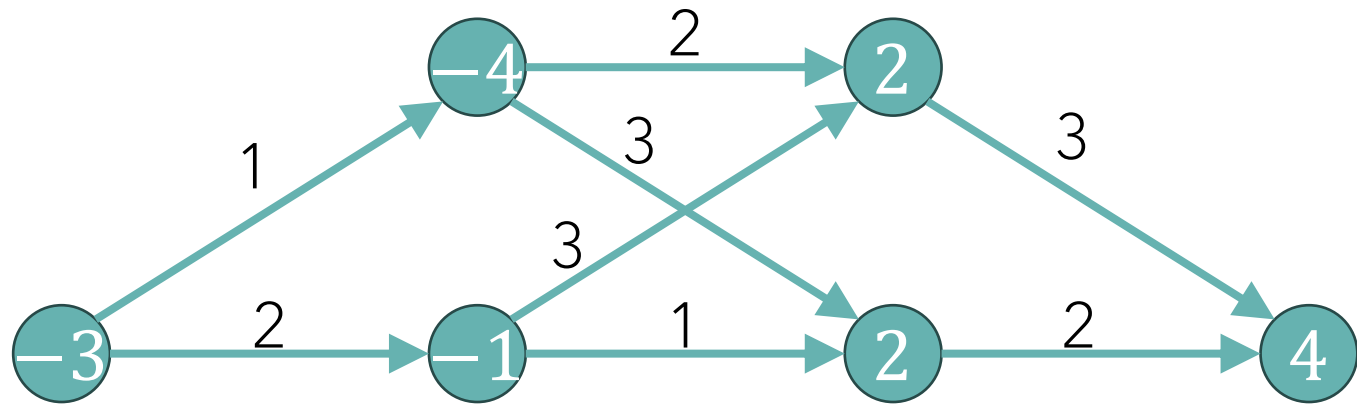


UPGRADING OUR FLOW NETWORKS

What if we relax the condition that we can only have a single source and a single sink?

New plan: Every node can be a source or a sink! (But not both.)

We will assign a *demand value* d_v to each node v . Negative demand indicates a supply.



UPGRADING OUR FLOW NETWORKS

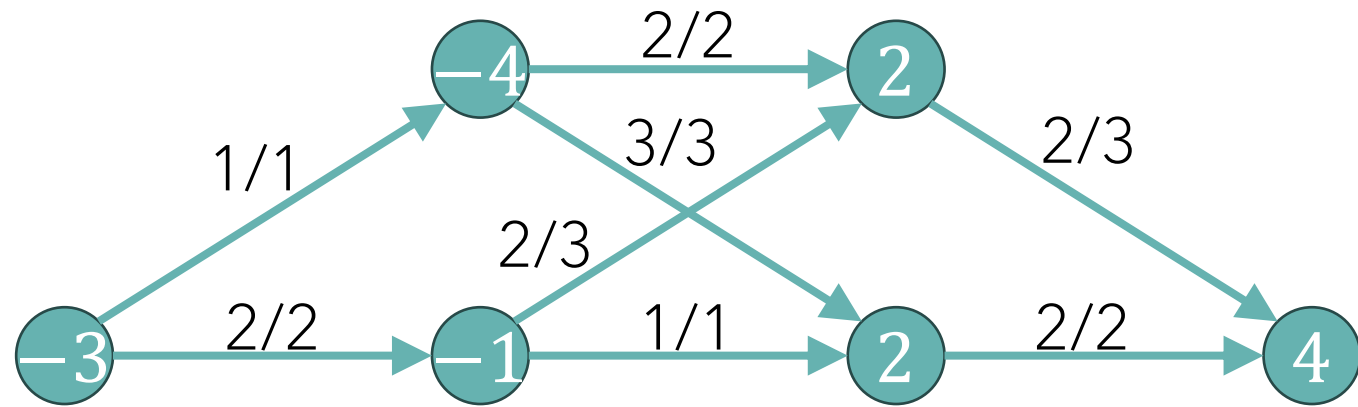
What if we relax the condition that we can only have a single source and a single sink?

New plan: Every node can be a source or a sink! (But not both.)

We will assign a *demand value* d_v to each node v . Negative demand indicates a supply.

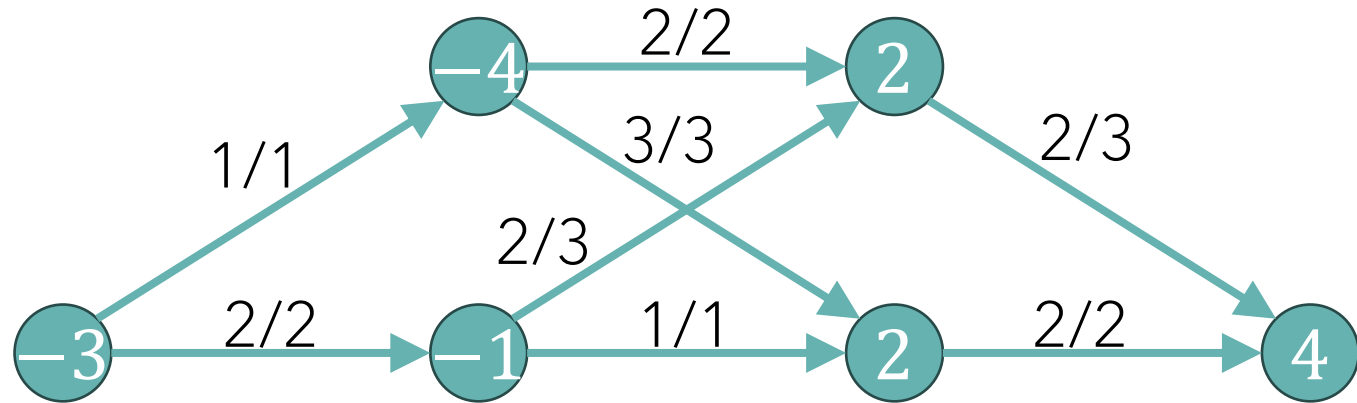
Our conservation condition needs to be updated: Now, $f_{in}(v) - f_{out}(v) = d_v$.

Our baseline question becomes a simple yes/no: Can we satisfy all of the demands?



CIRCULATIONS

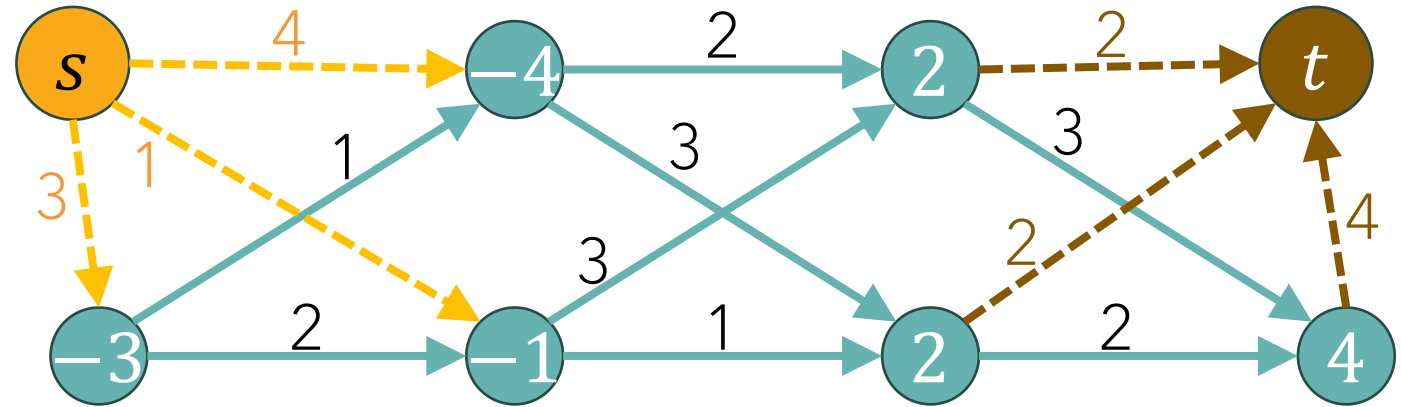
Our new problem is called a *circulation*.



An obvious necessary condition for a feasible circulation (one that satisfies all vertices) is that supply must equal demand. In other words: $\sum_{v \in V} d_v = 0$. This is easy to check.

Once we have verified this condition, though, how can we check if we can satisfy all demands and stay within capacity constraints?

ONE SIMPLE TRICK



We'll do the same thing yet again: Add a new source & sink!

Connect s to each supply node and t to each demand node, with capacities equal to their value.

Now we can just run ordinary MaxFlow. The circulation is feasible if and only if we can find a flow with value equal to the total supply.

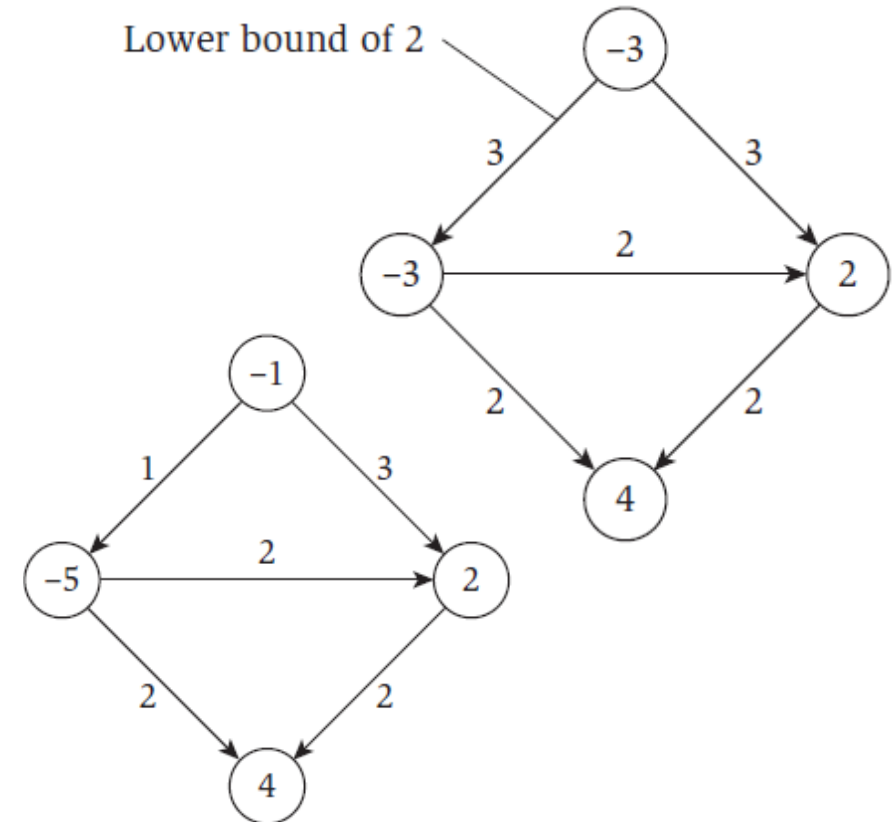
ANOTHER UPGRADE: MINIMUM USAGE

Let's add another type of constraint: some of our edges have a minimum capacity as well as a maximum. (Maybe we're under contract to use a certain supplier.)

How can we account for this?

The solution is surprisingly simple: Just take the minimum flow as a given and remove it from the problem entirely!

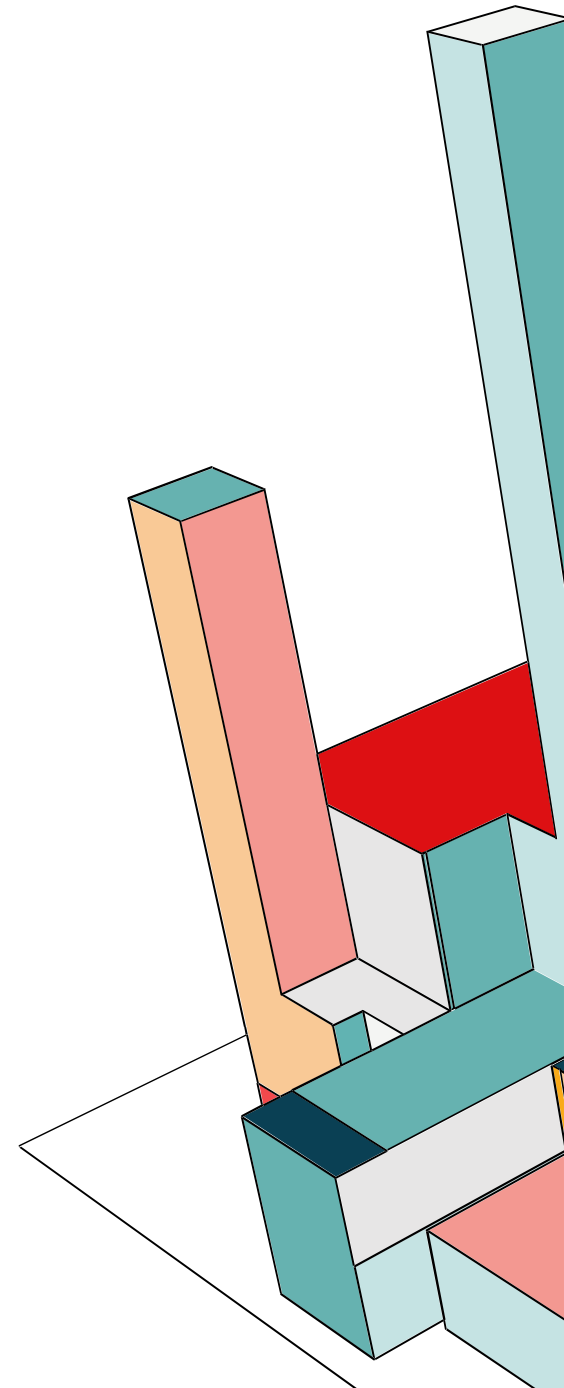
The edge has its capacity reduced, and the demand values for the nodes are adjusted accordingly. ("from" node goes up, "to" node goes down)



THE KEY POINT

Now that we have shown that all of these problems and variations are reducible to the Max-Flow Problem, we can model other problems as these generalizations – we don't need to map everything directly onto Max-Flow.

For example, if we have a problem that we can model as a circulation with minimum capacities, we know it is solvable using Ford-Fulkerson.



COOL APPLICATION: IMAGE SEGMENTATION

Given some image, can we determine what's in the foreground and what's in the background?

One approach: somehow assign each pixel a probability of being foreground vs. background.

Key restriction: Pixels that are near each other should tend to be in the same group.

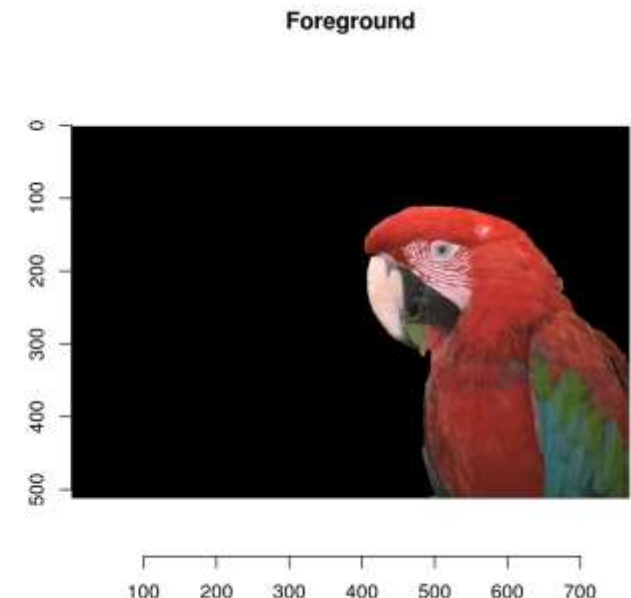
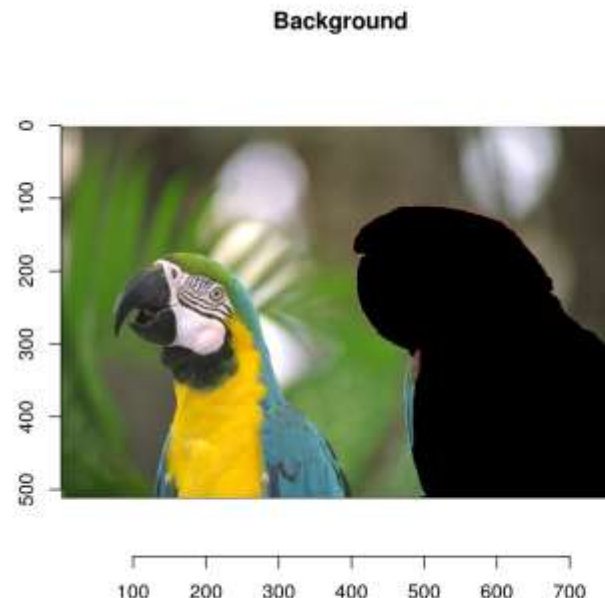


IMAGE SEGMENTATION VIA NETWORK FLOW

If we're going to make a network flow problem out of this, we need to produce a graph.

The nodes of our graph will be the pixels; the edges will represent the "nearby" relationship.

Since we're dividing up the pixels into two groups, it starts to feel like a MinCut problem.

What do we want to minimize, though?

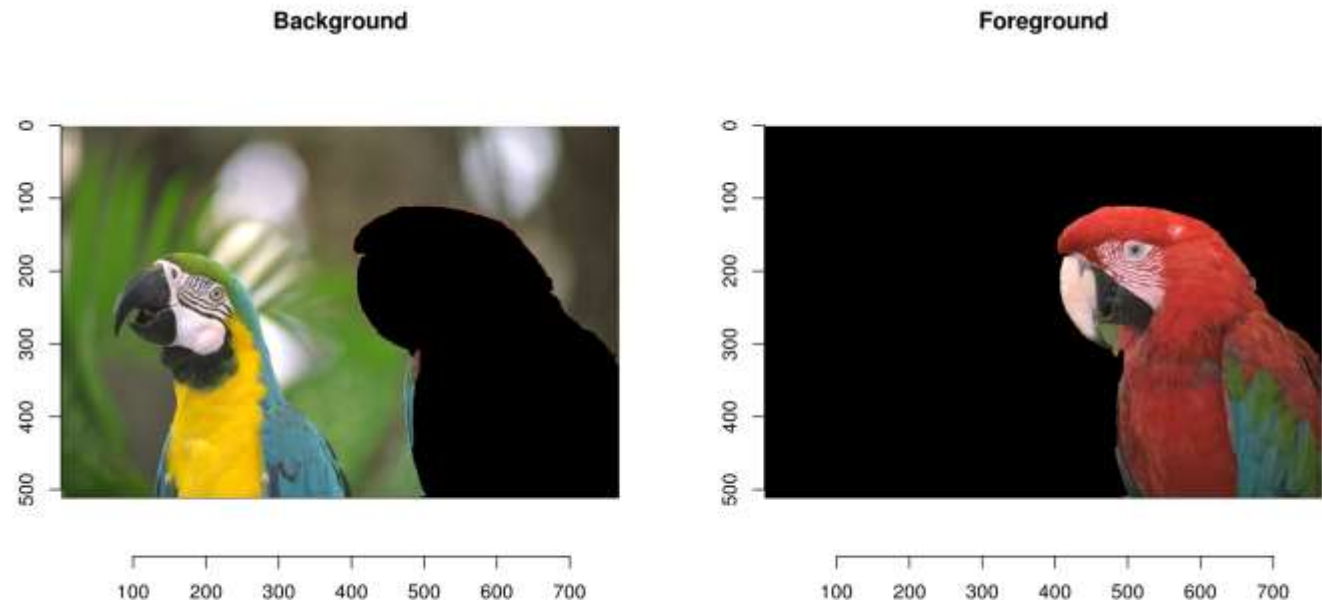


IMAGE SEGMENTATION VIA NETWORK FLOW

Many many problems create some sort of “loss” or “penalty” function that we want to minimize.

In this problems we have three things that we want to penalize:

- Putting a pixel in the foreground when it has a high probability of being in the background.
- The reverse situation (flipping foreground/background)
- Putting “nearby” pixels in different regions

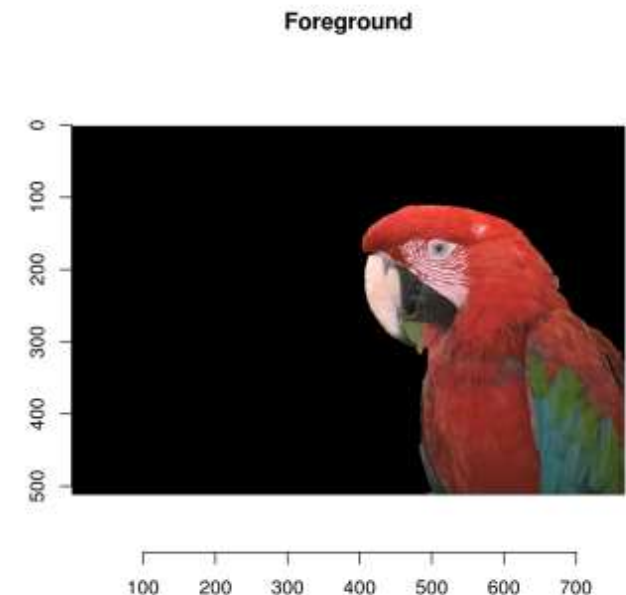
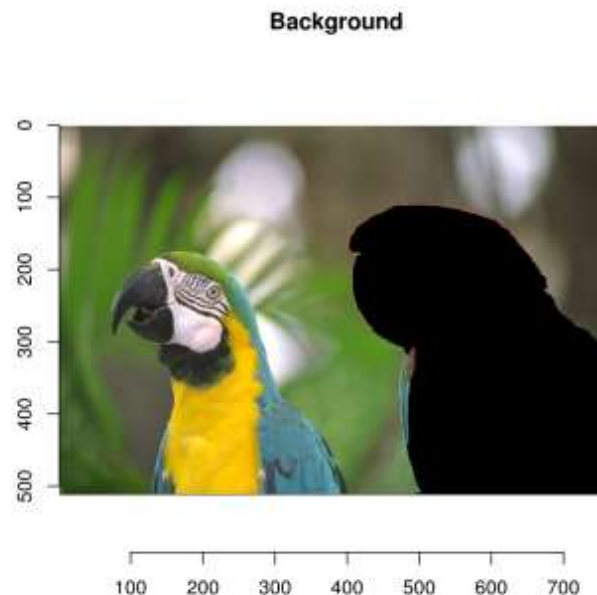


IMAGE SEGMENTATION VIA NETWORK FLOW

An obvious step is to assign to each edge of the “neighbor” graph the penalty p_{uv} for assigning nodes u and v differently.

The trick is how to handle the source & sink.

We will have the source represent the foreground; sink is the background.

We then connect the source to every node with a weight (penalty) equal to the *a priori* probability that it should go into the fore.

Do the same with the sink and the background probabilities.

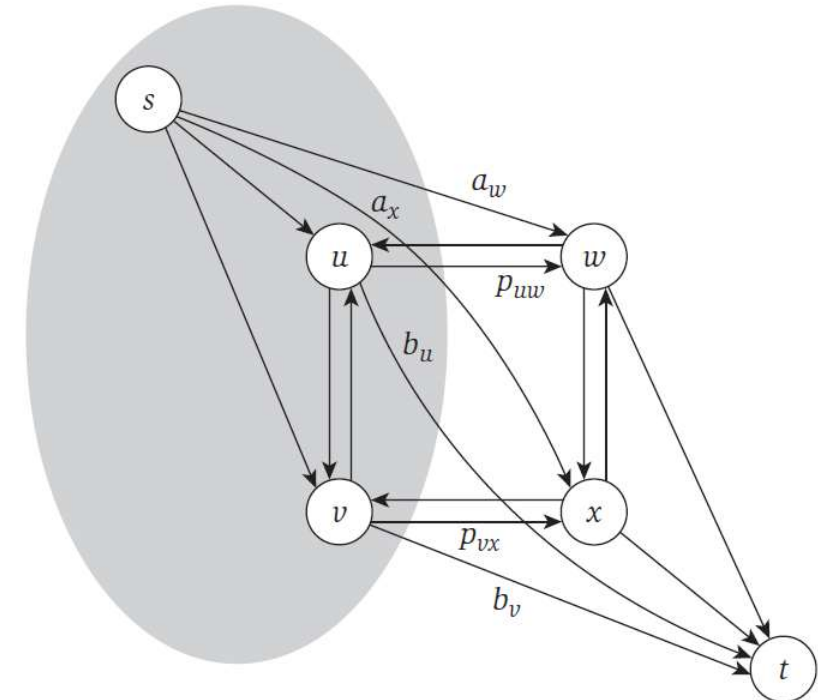
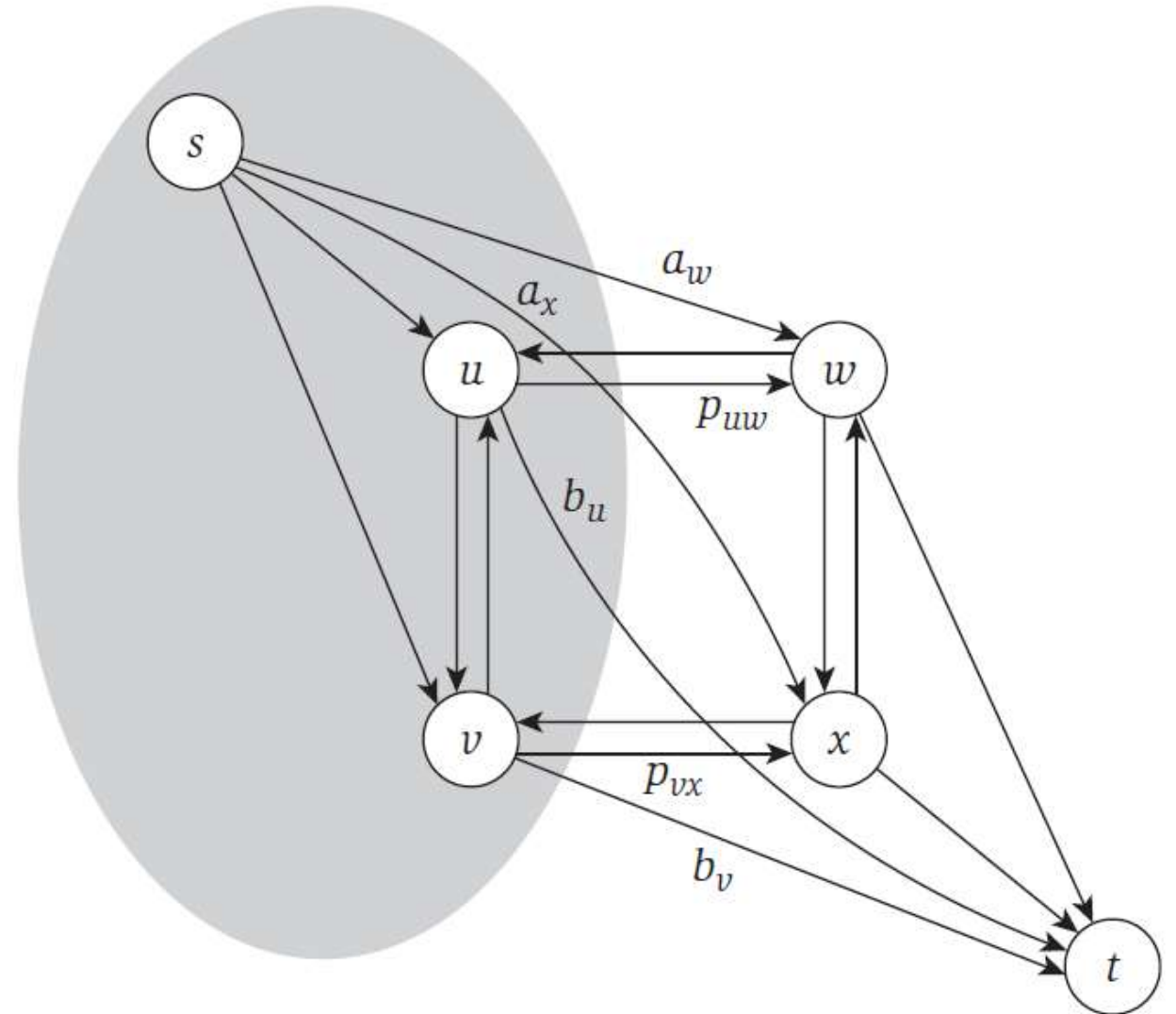
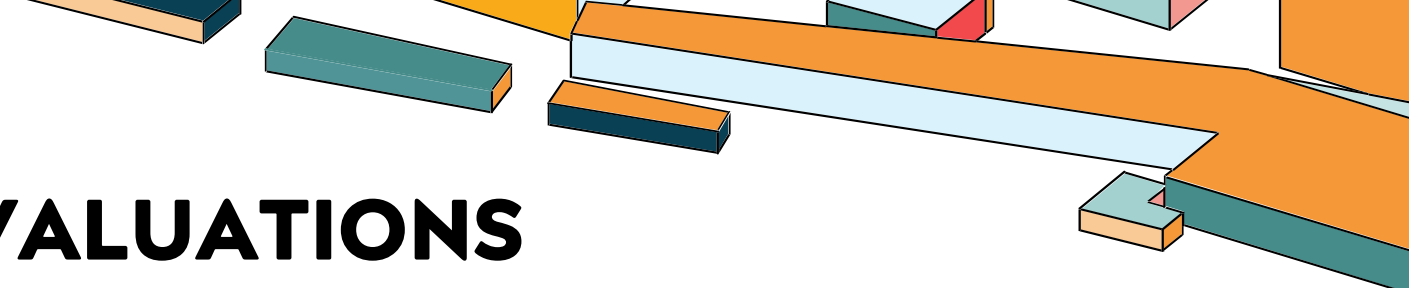


IMAGE SEGMENTATION VIA NETWORK FLOW

What we end up with is a situation where the penalties we would like to impose are precisely the edges that will cross the s - t cut.

Thus, we can minimize these penalties by producing a MinCut, which we know how to do.





INSTITUTE COURSE EVALUATIONS

- RPI's formal course evaluations will open next week and be available from 4/14 through 4/30.
- The department, school, and Institute take these seriously.
- The accuracy of the data depends upon a high participation rate.
- Thus, I will incentivize completion as follows:
 - There are 225 students in the course. 75% is around 170.
 - Each student beyond 170 that completes a course evaluation will result in everyone getting a half-point bonus on their final exam (which will be out of 200 points total).

ATTENDANCE POLL & FEEDBACK FORM

Please complete both.

